

CIFAR-10 Classification with VGG-A Variants and Batch Normalization

Student Name: *to be provided* Student ID: *to be provided*

Code: GitHub repository Dataset: CIFAR-10 website Trained weights: *to be uploaded*

Abstract

This report studies CIFAR-10 classification with VGG-A-style convolutional networks and focuses on the effect of Batch Normalization (BN) on performance and optimization. A plain VGG-A baseline, a BN variant, dropout and reduced-width variants, optimizer and activation ablations, and two loss-landscape visualizations are evaluated under a controlled protocol. The strongest model is VGG-A-BN with GELU activations, reaching 83.96% test accuracy. In matched ReLU experiments, BN improves the VGG-A baseline from 76.51% to 83.24% best test accuracy while adding only 5,504 parameters. Across learning-rate runs, BN reduces the mean per-step loss-envelope width from 0.4084 to 0.2147, indicating lower sensitivity to the tested step sizes.

1 Introduction

CIFAR-10 is small enough to permit repeated controlled experiments, yet it remains sensitive to optimization, model capacity, and regularization choices. This project uses a compact VGG-A architecture as the main testbed because it contains the required convolutional, pooling, activation, and fully connected components while remaining simple enough for direct ablation. The central question is not only which variant obtains the lowest test error, but also how BN changes the training behavior of the same base architecture.

The report therefore emphasizes experiment-level evidence. The introduction and implementation description are kept brief; most space is used for quantitative comparisons, curve-based observations, and the optimization analysis required by the assignment.

2 Method

2.1 Data and Training Protocol

All experiments use the CIFAR-10 training split for optimization and the CIFAR-10 test split for validation/test reporting. Images are converted to tensors and normalized channel-wise with mean $(0.5, 0.5, 0.5)$ and standard deviation $(0.5, 0.5, 0.5)$. Unless stated otherwise, models are trained for 20 epochs with batch size 128, Adam, learning rate 10^{-3} , weight decay 0, cross-entropy loss, and random seed 2020 on `cuda:2`. The reported accuracy is the best test accuracy observed during the 20 epochs, and test error is $1 - \text{accuracy}$.

2.2 Architectures

The baseline is a VGG-A-style network adapted to 32×32 inputs. It uses five convolutional stages, max pooling, ReLU nonlinearities, and a three-layer classifier with two hidden linear layers of width 512. The BN variant inserts BatchNorm2d after each convolution and before the activation. Additional v strongly reduced-width mo GELU. These variants cover filters/neurons, regularization choices.

[illegible]

Figure 1: VGG-style feature extractor used as the project baseline.

the activation. Additional variants test classifier dropout, a strongly reduced-width model, AdamW, LeakyReLU, and GELU. These variants cover the assignment requirements on filters/neurons, regularization, activation functions, and optimizer choices.

3 Main Results

3.1 Overall Model Comparison

Table 1 summarizes the principal runs. The plain VGG-A baseline reaches 76.51% best test accuracy but ends with a large train-test gap: 96.80% final train accuracy against 74.11% final test accuracy. Adding BN raises the best test accuracy to 83.24%, reduces the best test error to 16.76%, and reaches this best epoch earlier. The parameter cost is negligible relative to the model size: 9,756,426 parameters for BN versus 9,750,922 for the baseline.

Figure 2 shows why the BN improvement is not merely a final-epoch artifact. Under the same optimizer, learning rate, batch size, seed, and preprocessing, the BN curve moves to a higher validation-accuracy regime earlier and remains there. The baseline improves during the early epochs but then shows increasing validation loss, which is consistent with the overfitting visible in its final train-test gap. BN does not eliminate overfitting, but it changes the optimization trajectory enough to reach a better solution within the same epoch budget.

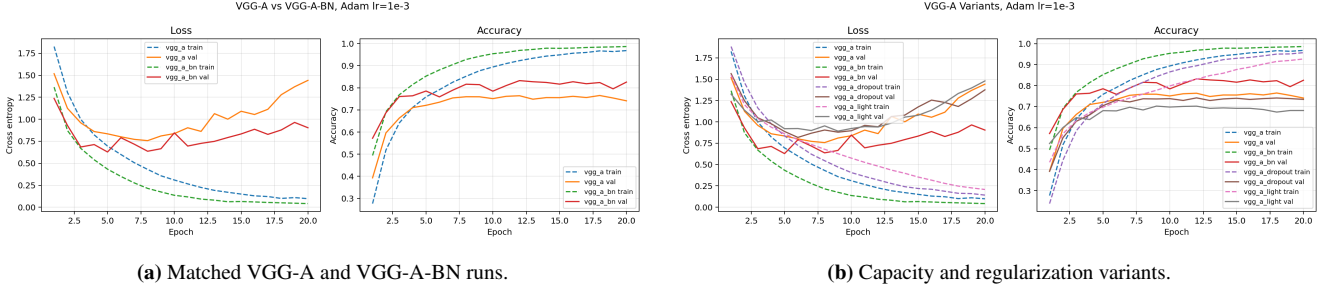
3.2 Capacity and Regularization

The light model reduces the parameter count to 285,162, only 2.92% of the full VGG-A size. This makes it faster, with 56.06 seconds of recorded epoch time compared with 145.52 seconds for the baseline run, but it loses 6.24 percentage points relative to VGG-A and 12.97 percentage points relative to VGG-A-BN. The result indicates that the full VGG-A has useful capacity for CIFAR-10 under the chosen training budget. The light model still overfits after its best epoch, so reducing width alone is not a sufficient regularizer.

Classifier dropout also underperforms in this setting.

Table 1: Main CIFAR-10 results. All runs use 20 epochs and batch size 128.

Model	Optimizer	Params	Best Acc.	Best Err.	Best Epoch	Final Acc.
VGG-A	Adam	9,750,922	0.7651	0.2349	18	0.7411
VGG-A-BN	Adam	9,756,426	0.8324	0.1676	12	0.8257
VGG-A-Dropout	Adam	9,750,922	0.7419	0.2581	12	0.7348
VGG-A-Light	Adam	285,162	0.7027	0.2973	9	0.6818
VGG-A-BN	AdamW	9,756,426	0.8302	0.1698	20	0.8302
VGG-A-BN-LeakyReLU	Adam	9,756,426	0.8254	0.1746	18	0.8194
VGG-A-BN-GELU	Adam	9,756,426	0.8396	0.1604	19	0.8373

**Figure 2:** Training curves for the baseline, BN, dropout, and light models.

VGG-A-Dropout reaches 74.19% best accuracy, lower than the plain VGG-A baseline. It reduces final train accuracy slightly, but this reduction does not translate into better validation accuracy. The likely reason is that dropout is applied only in the classifier, while much of the useful inductive bias and optimization improvement in this architecture comes from the convolutional feature extractor and BN-normalized intermediate activations.

3.3 Optimizer and Activation Ablations

Adam and AdamW are close on VGG-A-BN. Adam reaches a slightly higher best accuracy, 83.24% versus 83.02%, and does so earlier. AdamW, however, has the stronger final epoch: 83.02% final accuracy and 0.8285 final validation loss, compared with 82.57% and 0.9035 for Adam. This suggests that decoupled weight decay smooths the final state, but the 20-epoch budget is not long enough to convert that into a higher peak accuracy.

The activation comparison is more decisive. With the same BN architecture and Adam setting, GELU obtains 83.96% best accuracy and 83.73% final accuracy, outperforming ReLU and LeakyReLU. LeakyReLU does not improve peak accuracy over ReLU, although its final validation loss is lower than the ReLU baseline. GELU’s advantage is consistent with a smoother nonlinear transformation, but the experiment should be read empirically: in this architecture and training budget, GELU is the best tested activation.

4 Batch Normalization and Optimization

4.1 Matched BN Comparison

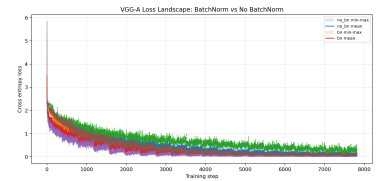
BN improves the matched VGG-A experiment in three measurable ways. First, it improves accuracy by 6.73 percentage points. Second, it reaches its best result at epoch 12 rather than epoch 18, which indicates faster useful convergence. Third, it reduces final validation loss from 1.4421

to 0.9035. Since the architecture, data preprocessing, optimizer, batch size, learning rate, and seed are held fixed, the improvement is attributable to the BN layers and their effect on the optimization process rather than to increased model capacity.

4.2 Learning-Rate Loss Envelope

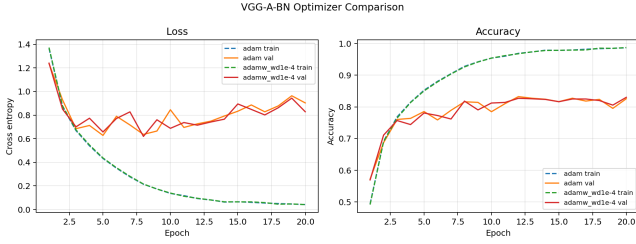
To test BN from an optimization-landscape perspective, I trained VGG-A and VGG-A-BN with learning rates 10^{-4} , 5×10^{-4} , 10^{-3} , and 2×10^{-3} . During training, the loss at every update step was saved. For each model family and each step index, I computed the minimum and maximum loss across the four learning-rate runs and plotted the filled region between the two curves. This plot measures variation of the loss trajectory across step-size choices; it should not be confused with geometric sharpness in parameter space.

The resulting envelope is shown in Fig. 4. The mean envelope width is 0.4084 for no-BN and 0.2147 for BN, a 47.4% reduction. The final loss range

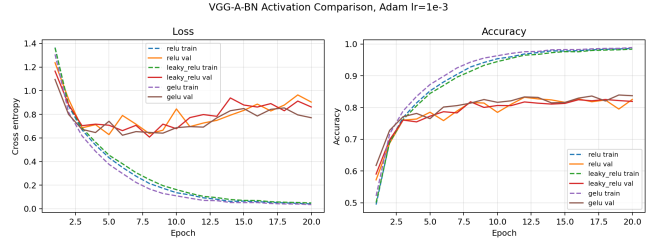
**Figure 4:** Per-step loss envelopes across four learning rates.

is also much narrower for BN: 0.0306–0.0551 compared with 0.0576–0.2764 for no-BN. These numbers support a specific claim: BN reduces the variation of the training loss trajectory across the tested learning rates and therefore makes the model less sensitive to step-size selection after the early phase of training.

The maximum instantaneous envelope width is larger for BN, 3.5587 versus 2.7904. This does not contradict the main conclusion because the maximum is dominated by a transient spread: the BN run at 10^{-4} converges slowly while the larger learning rates improve much more quickly. The average and final envelopes are more representative of step-size robustness over the training process.



(a) Adam and AdamW on VGG-A-BN.



(b) ReLU, LeakyReLU, and GELU on VGG-A-BN.

Figure 3: Optimizer and activation ablations on the BN architecture.

Table 2: Learning-rate sweep for VGG-A and VGG-A-BN.

Model	LR	Best Acc.	Final Acc.	Final Loss
VGG-A	10^{-4}	0.7659	0.7526	1.3084
VGG-A	5×10^{-4}	0.7840	0.7801	1.2380
VGG-A	10^{-3}	0.7651	0.7411	1.4421
VGG-A	2×10^{-3}	0.7289	0.7179	1.1336
VGG-A-BN	10^{-4}	0.7453	0.7203	1.6119
VGG-A-BN	5×10^{-4}	0.8244	0.8244	0.8311
VGG-A-BN	10^{-3}	0.8324	0.8257	0.9035
VGG-A-BN	2×10^{-3}	0.8305	0.8192	0.8693

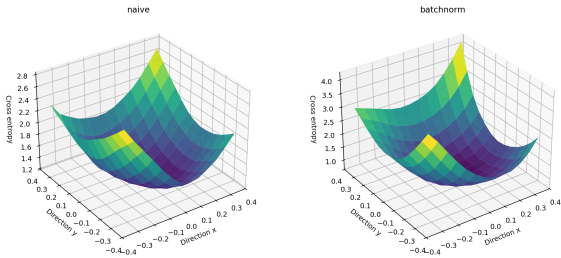


Figure 5: Local 3D validation-loss surfaces around the trained VGG-A and VGG-A-BN checkpoints. The plot uses two random normalized parameter-space directions, a 13×13 grid with radius 0.35, and 1,024 validation examples.

4.3 Local 3D Surface Visualization

Figure 5 gives an additional qualitative view. Around the trained checkpoints, the center loss is 1.2258 for the naive model and 0.7133 for the BN model on the sampled validation subset. The plot is useful as a visual comparison of the local neighborhoods, but it is not used as the primary sharpness metric because random two-dimensional slices can vary with the chosen directions. The learning-rate envelope above is therefore the stronger quantitative evidence for BN’s improved step-size robustness.

5 Discussion

The best-performing model is VGG-A-BN with GELU, with a best test error of 16.04%. The strongest architectural conclusion is that BN is more valuable than classifier-only dropout or simple capacity reduction in this experimental setting. The light model shows that parameter efficiency can be improved dramatically, but its loss in accuracy is too large for it to be the main submission model. AdamW produces a better final state than Adam but not a better peak within 20 epochs, so longer training or a schedule would be the next optimization direction.

The BN experiments also clarify terminology. A narrower learning-rate loss envelope does not prove that the loss surface is geometrically flatter in the Hessian sense. It shows that the training loss trajectory is less dispersed across the

tested step sizes. This is still directly relevant to optimization: a method that tolerates a wider range of useful learning rates is easier to train and less dependent on precise hyperparameter tuning.

6 Conclusion

The experiments satisfy the required architectural, regularization, activation, optimizer, and visualization components. BN is the most important tested modification to VGG-A: it improves the matched baseline by 6.73 percentage points, improves convergence speed, and reduces the average per-step loss-envelope width across learning rates by 47.4%. GELU on top of BN gives the best final model, reaching 83.96% test accuracy. The report therefore supports BN both as a performance-improving architectural component and as an optimization stabilizer for this CIFAR-10 VGG-A setting.

References

- [1] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” 2009.
- [2] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv:1412.6980, 2014.
- [3] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How Does Batch Normalization Help Optimization?” NeurIPS, 2018.
- [4] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” ICLR, 2015.